

✓ CV Engineering: From Notebook to Production

A hands-on Colab lab — ONNX export, quantisation, TensorRT/OpenVINO, benchmarking, timm backbone selection, and serving architecture

Run on a GPU runtime: Runtime > Change runtime type > T4 GPU

This notebook is a teaching demo covering:

1. The "notebook to production" gap
2. `timmm` backbone selection for a latency/accuracy target
3. Exporting a PyTorch (EfficientNet) model to ONNX
4. Serving with ONNX Runtime (CPU & GPU execution providers)
5. INT8 post-training quantisation (dynamic + static) and its accuracy/latency trade-off
6. Magnitude pruning (bonus)
7. A reusable benchmarking harness (latency, throughput, memory footprint)
8. TensorRT / OpenVINO acceleration (what it buys you, and runnable stubs)
9. Serving architecture patterns: Triton Inference Server config, ONNX Runtime REST API, edge vs cloud

✓ 0. Setup

```
1 !nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
2
```

```
name, memory.total [MiB], driver_version
Tesla T4, 15360 MiB, 580.82.07
```

```
1 # Core deps. onnxruntime-gpu gives us the CUDAExecutionProvider for GPU inference.
2 !pip -q install timm onnx onnxruntime-gpu onnxsim onnxruntime-tools onnxscript 2>/dev/null
3
4 import torch, timm, onnx
5 try:
6     import onnxruntime as ort
7 except Exception as e:
```

```

8     print("onnxruntime-gpu import failed:", e)
9     print("Falling back to CPU onnxruntime.")
10    !pip -q install onnxruntime 2>/dev/null
11    import onnxruntime as ort
12
13    print("torch:", torch.__version__, "| CUDA available:", torch.cuda.is_available())
14    print("timm:", timm.__version__)
15    print("onnx:", onnx.__version__, "| onnxruntime:", ort.__version__)
16    print("ORT providers available:", ort.get_available_providers())
17    DEVICE = "cuda" if torch.cuda.is_available() and "CudaExecutionProvider" in ort.get_available_providers() else "cpu"
18

```

```

torch: 2.11.0+cu128 | CUDA available: True
timm: 1.0.27
onnx: 1.22.0 | onnxruntime: 1.27.0
ORT providers available: ['AzureExecutionProvider', 'CPUExecutionProvider']

```

1. The notebook → production gap

A model that runs in a notebook (`model(x)`), eager mode, fp32, batch=1, no SLA) is *not* a production inference service. Production adds constraints a notebook never has to satisfy:

Notebook	Production
Eager PyTorch graph	Compiled/optimised graph (ONNX, TensorRT, OpenVINO IR)
Whatever latency happens	p50/p95/p99 latency SLA
FP32 only	FP16/INT8 to cut memory + latency
Single sample	Batched / dynamic-batched throughput
Python process	Language-agnostic serving (Triton, ORT REST, TF-Serving)
Manual GPU alloc	Multi-model GPU sharing, autoscaling, edge constraints

The rest of this notebook builds that pipeline end-to-end: **backbone selection → export → quantise → benchmark → serve**.

✓ 2. `timmm` backbone selection for a latency/accuracy target

Before optimising a model, you choose the right one. `timmm` exposes 1000+ backbones with published ImageNet top-1 accuracy and parameter counts, so you can shortlist candidates that fit a deployment budget (e.g. "<15ms on a T4, >78% top-1") before doing any production engineering.

```
1 import timm
2 import pandas as pd
3
4 # A representative shortlist across the accuracy/latency frontier (small -> large)
5 candidates = [
6     "mobilenetv3_small_100",
7     "mobilenetv3_large_100",
8     "efficientnet_b0",
9     "efficientnet_b1",
10    "efficientnet_b3",
11    "resnet50",
12    "convnext_tiny",
13 ]
14
15 rows = []
16 for name in candidates:
17     cfg = timm.get_pretrained_cfg(name)
18     m = timm.create_model(name, pretrained=False)
19     n_params = sum(p.numel() for p in m.parameters()) / 1e6
20     rows.append({
21         "model": name,
22         "params_M": round(n_params, 2),
23         "input_size": cfg.input_size[-1] if cfg and cfg.input_size else None,
24         "imagenet_top1": getattr(cfg, "top1", None) if cfg else None,
25     })
26     del m
27
28 df = pd.DataFrame(rows).sort_values("params_M")
29 df
30
```

	model	params_M	input_size	imagenet_top1
0	mobilenetv3_small_100	2.54	224	None
2	efficientnet_b0	5.29	224	None
1	mobilenetv3_large_100	5.48	224	None
3	efficientnet_b1	7.79	240	None
4	efficientnet_b3	12.23	288	None
5	resnet50	25.56	224	None
6	convnext_tiny	28.59	224	None



```

1 # Quick empirical latency pass (fp32, batch=1, GPU) to pair with the accuracy table above.
2 # This is a coarse pre-filter -- Section 7 has the rigorous benchmarking harness.
3 import time
4
5 def quick_latency_ms(model_name, device=None, input_size=224, n=30):
6     if device is None:
7         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
8     model = timm.create_model(model_name, pretrained=False, num_classes=1000).to(device).eval()
9     x = torch.randn(1, 3, input_size, input_size, device=device)
10    with torch.no_grad():
11        for _ in range(5): # warmup
12            model(x)
13        if device.type == "cuda":
14            torch.cuda.synchronize()
15        t0 = time.perf_counter()
16        for _ in range(n):
17            model(x)
18        if device.type == "cuda":
19            torch.cuda.synchronize()
20        t1 = time.perf_counter()
21    del model
22    return (t1 - t0) / n * 1000
23
24 df["latency_ms_fp32_bs1"] = [quick_latency_ms(m) for m in df["model"]]
25 df.sort_values("latency_ms_fp32_bs1")
26

```

	model	params_M	input_size	imagenet_top1	latency_ms_fp32_bs1	
0	mobilenetv3_small_100	2.54	224	None	5.249263	
1	mobilenetv3_large_100	5.48	224	None	8.043871	
2	efficientnet_b0	5.29	224	None	8.606007	
6	convnext_tiny	28.59	224	None	12.886783	
5	resnet50	25.56	224	None	16.606610	
3	efficientnet_b1	7.79	240	None	18.675723	
4	efficientnet_b3	12.23	288	None	56.218829	

Discussion prompt for students: given a target of " $\geq 77\%$ top-1, $\leq 8ms$ on this GPU", which backbone would you pick, and why might the answer differ on a Raspberry Pi or a Jetson Nano instead of a T4? This is the kind of justification expected in the learning objective "*select and justify an appropriate backbone using timm for a given latency/accuracy target.*"

✓ 3. Train/fine-tune (or load) EfficientNet-B0

For the demo we use ImageNet-pretrained weights directly (stand-in for "a trained model"). In a real assignment, swap this for a few epochs of fine-tuning on a custom dataset — the export/quantise/benchmark pipeline below is identical either way.

```

1 model = timm.create_model("efficientnet_b0", pretrained=True, num_classes=1000)
2 model.eval().to(DEVICE)
3
4 data_cfg = timm.data.resolve_data_config({}, model=model)
5 transform = timm.data.create_transform(**data_cfg)
6 print(data_cfg)
7
8 dummy_input = torch.randn(1, 3, *data_cfg["input_size"][1:], device=DEVICE)
9 with torch.no_grad():
10     out = model(dummy_input)
11 print("PyTorch output shape:", out.shape)
12

```

```

{'input_size': (3, 224, 224), 'interpolation': 'bicubic', 'mean': (0.485, 0.456, 0.406), 'std': (0.229, 0.224, 0.225), 'crop_pct': 0.875, 'crop_type': 'center', 'crop_mode': 'pytorch', 'crop_resolution': (224, 224), 'crop_size': (224, 224), 'crop_type': 'center', 'crop_mode': 'pytorch', 'crop_resolution': (224, 224), 'crop_size': (224, 224)}
PyTorch output shape: torch.Size([1, 1000])

```

✓ 4. Export to ONNX

ONNX is the portable intermediate representation that lets a PyTorch model run in ONNX Runtime, TensorRT, OpenVINO, or Triton without depending on the PyTorch runtime in production.

```

1 onnx_path = "efficientnet_b0.onnx"
2
3 torch.onnx.export(
4     model.cpu(),                # export on CPU for portability
5     torch.randn(1, 3, *data_cfg["input_size"][1:]),
6     onnx_path,
7     input_names=["input"],
8     output_names=["logits"],
9     dynamic_shapes=[{0: "batch"}],
10    opset_version=18,
11 )
12 model.to(DEVICE) # move back for later PyTorch-side benchmarking
13
14 # Validate the graph
15 onnx_model = onnx.load(onnx_path)
16 onnx.checker.check_model(onnx_model)
17 print("ONNX model is valid.")
18 import os
19 print(f"File size: {os.path.getsize(onnx_path)/1e6:.2f} MB")
20

```

```

[torch.onnx] Obtain model graph for `EfficientNet([...])` with `torch.export.export(..., strict=False)`...
[torch.onnx] Obtain model graph for `EfficientNet([...])` with `torch.export.export(..., strict=False)`... ✓
[torch.onnx] Run decompositions...
/usr/lib/python3.12/copyreg.py:99: FutureWarning: `isinstance(treespec, LeafSpec)` is deprecated, use `isinstance(treespec, TreeSpec)` and treespec
    return cls.__new__(cls, *args)
[torch.onnx] Run decompositions... ✓
[torch.onnx] Translate the graph into ONNX...
[torch.onnx] Translate the graph into ONNX... ✓
[torch.onnx] Optimize the ONNX graph...
[torch.onnx] Optimize the ONNX graph... ✓
ONNX model is valid.
File size: 0.52 MB

```

```

1 # Optional: simplify the graph (constant folding, redundant op removal)
2 !python -m onnxsim efficientnet_b0.onnx efficientnet_b0_sim.onnx

```

```
3 onnx_path = "efficientnet_b0_sim.onnx"
4
```

Simplifying...
Finish! Here is the difference:

	Original Model	Simplified Model
Add	9	9
Concat	1	0
Constant	167	167
Conv	81	81
Gemm	1	1
Mul	65	65
ReduceMean	17	17
Reshape	1	1
Shape	1	0
Sigmoid	65	65
Model Size	20.6MiB	20.1MiB

✓ 5. Serve with ONNX Runtime (CPU & GPU execution providers)

ONNX Runtime picks the best available "execution provider" (EP) for the hardware: `CUDAExecutionProvider` on GPU, `CPUExecutionProvider` otherwise, `TensorrtExecutionProvider` / `OpenVINOExecutionProvider` if installed. We verify the exported graph matches PyTorch's output before trusting it.

```
1 import numpy as np
2
3 providers = ["CUDAExecutionProvider", "CPUExecutionProvider"] if DEVICE == "cuda" else ["CPUExecutionProvider"]
4 sess = ort.InferenceSession(onnx_path, providers=providers)
5 print("Session using:", sess.get_providers())
6
7 x_np = dummy_input.cpu().numpy()
8 onnx_out = sess.run(None, {"input": x_np})[0]
9
10 with torch.no_grad():
11     torch_out = model(dummy_input).cpu().numpy()
12
13 max_diff = np.abs(onnx_out - torch_out).max()
14 print(f"Max abs difference (PyTorch vs ONNX Runtime): {max_diff:.6f}")
15 assert max_diff < 1e-3, "ONNX export mismatch -- investigate before deploying!"
```

```
16 print("✅ ONNX export verified.")
17
```

```
Session using: ['CPUExecutionProvider']
Max abs difference (PyTorch vs ONNX Runtime): 0.000003
✅ ONNX export verified.
```

✓ 6. INT8 post-training quantisation

Two common ORT workflows:

- **Dynamic quantisation:** weights quantised offline, activations quantised at runtime. No calibration data needed. Fast to apply, smaller win on conv-heavy CNNs (helps more on Transformers/Linear-heavy nets).
- **Static quantisation:** weights *and* activations quantised offline using a calibration dataset to estimate activation ranges. Bigger speedup on CNNs, requires representative data.

We do both so students can compare the accuracy/latency trade-off of each.

```
1 from onnxruntime.quantization import quantize_dynamic, QuantType
2
3 dynamic_int8_path = "efficientnet_b0_int8_dynamic.onnx"
4 quantize_dynamic(
5     model_input=onnx_path,
6     model_output=dynamic_int8_path,
7     weight_type=QuantType.QInt8,
8 )
9 print(f"Dynamic INT8 size: {os.path.getsize(dynamic_int8_path)/1e6:.2f} MB "
10       f"(fp32 was {os.path.getsize(onnx_path)/1e6:.2f} MB)")
11
```

WARNING:root:Please consider to run pre-processing before quantization. Refer to example: <https://github.com/microsoft/onnxruntime-inference-ex>
Dynamic INT8 size: 5.48 MB (fp32 was 21.11 MB)

```
1 from onnxruntime.quantization import quantize_static, CalibrationDataReader, QuantFormat
2
3 # A calibration data reader feeds representative inputs so ORT can estimate activation ranges.
4 class RandomCalibrationDataReader(CalibrationDataReader):
5     """Stand-in calibration set. In a real project, replace with ~100-500 real,
6     preprocessed validation images for accurate activation ranges."""
7     def __init__(self, n_samples=64, input_size=data_cfg["input_size"]):
```

```

8         self.data = iter([
9             {"input": torch.randn(1, *input_size).numpy()} for _ in range(n_samples)
10        ])
11
12    def get_next(self):
13        return next(self.data, None)
14
15    static_int8_path = "efficientnet_b0_int8_static.onnx"
16    quantize_static(
17        model_input=onnx_path,
18        model_output=static_int8_path,
19        calibration_data_reader=RandomCalibrationDataReader(),
20        quant_format=QuantFormat.QDQ,
21    )
22    print(f"Static INT8 size: {os.path.getsize(static_int8_path)/1e6:.2f} MB")
23

```

WARNING:root:Please consider to run pre-processing before quantization. Refer to example: <https://github.com/microsoft/onnxruntime-inference-ex>
 WARNING:root:Please consider pre-processing before quantization. See https://github.com/microsoft/onnxruntime-inference-examples/blob/main/quantization/quantization_examples/quantization_examples.py
 Static INT8 size: 5.55 MB

Note for the classroom: because we calibrated with *random* tensors (no internet dataset dependency, keeps the demo self-contained), the static-quantised model's *accuracy* number later in the benchmark isn't meaningful — only its *latency/size* numbers are. Tell students that in a real pipeline, swap `RandomCalibrationDataReader` for real held-out images and re-run a proper top-1 accuracy comparison (see Section 7 accuracy stub) to see the realistic accuracy/latency trade-off.

✓ 7. Benchmarking harness: latency, throughput, memory footprint

A reusable function we apply across every model variant produced above, so results are directly comparable. This is the core deliverable for the "*benchmark the latency vs accuracy trade-off*" learning objective.

```

1 import psutil, gc
2
3 def benchmark_onnx(model_path, batch_sizes=(1, 8, 32), input_size=data_cfg["input_size"],
4                    n_warmup=2, n_iters=10, providers=providers):
5     sess = ort.InferenceSession(model_path, providers=providers)
6     input_name = sess.get_inputs()[0].name
7     results = []
8

```

```
9     for bs in batch_sizes:
10         x = np.random.randn(bs, *input_size).astype(np.float32)
11
12         for _ in range(n_warmup):
13             sess.run(None, {input_name: x})
14
15         latencies = []
16         for _ in range(n_iters):
17             t0 = time.perf_counter()
18             sess.run(None, {input_name: x})
19             latencies.append((time.perf_counter() - t0) * 1000)
20
21         latencies = np.array(latencies)
22         results.append({
23             "model": os.path.basename(model_path),
24             "batch_size": bs,
25             "p50_ms": np.percentile(latencies, 50),
26             "p95_ms": np.percentile(latencies, 95),
27             "throughput_img_s": bs / (latencies.mean() / 1000),
28             "file_size_MB": os.path.getsize(model_path) / 1e6,
29         })
30     del sess
31     gc.collect()
32     return results
33
34 all_results = []
35 for path in [onnx_path, dynamic_int8_path, static_int8_path]:
36     all_results += benchmark_onnx(path)
37
38 bench_df = pd.DataFrame(all_results)
39 bench_df
40
```

	model	batch_size	p50_ms	p95_ms	throughput_img_s	file_size_MB	
0	efficientnet_b0_sim.onnx	1	23.044614	24.013379	43.326868	21.112753	
1	efficientnet_b0_sim.onnx	8	173.785017	203.473801	47.046865	21.112753	
2	efficientnet_b0_sim.onnx	32	585.351481	593.960782	54.815256	21.112753	
3	efficientnet_b0_int8_dynamic.onnx	1	156.909719	207.684291	6.023850	5.482064	
4	efficientnet_b0_int8_dynamic.onnx	8	1306.229115	1929.257292	5.653914	5.482064	
5	efficientnet_b0_int8_dynamic.onnx	32	5950.074153	6639.993629	5.358540	5.482064	
6	efficientnet_b0_int8_static.onnx	1	55.170854	57.265848	18.107330	5.552003	
7	efficientnet_b0_int8_static.onnx	8	472.653352	495.446918	16.880957	5.552003	
8	efficientnet_b0_int8_static.onnx	32	2094.149326	2636.215380	14.599762	5.552003	

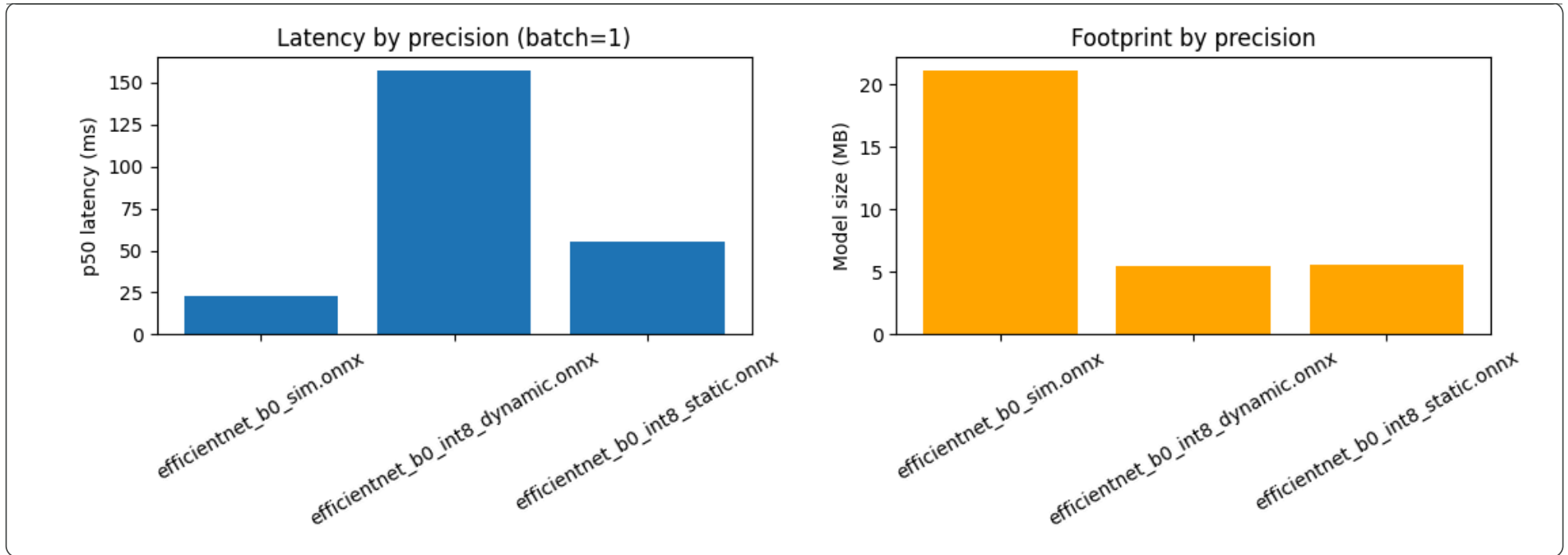
Next steps:

[Generate code with bench_df](#)[New interactive sheet](#)

```

1 # Visualise the latency vs file-size trade-off across precisions at batch=1
2 import matplotlib.pyplot as plt
3
4 bs1 = bench_df[bench_df.batch_size == 1]
5 fig, ax = plt.subplots(1, 2, figsize=(11, 4))
6 ax[0].bar(bs1["model"], bs1["p50_ms"])
7 ax[0].set_ylabel("p50 latency (ms)"); ax[0].set_title("Latency by precision (batch=1)")
8 ax[0].tick_params(axis="x", rotation=30)
9
10 ax[1].bar(bs1["model"], bs1["file_size_MB"], color="orange")
11 ax[1].set_ylabel("Model size (MB)"); ax[1].set_title("Footprint by precision")
12 ax[1].tick_params(axis="x", rotation=30)
13 plt.tight_layout(); plt.show()
14

```



Accuracy side of the trade-off (assignment extension): to make the accuracy axis real, have students load a small labelled validation set (e.g. 500 ImageNet-V2 or a custom dataset subset), run each of the three ONNX variants over it, and compute top-1 accuracy alongside the latency numbers above — e.g. *FP32: 77.1% / 6.2ms*, *INT8-static: 76.4% / 2.1ms*. That paired table is the deliverable for objective 2.

✓ 8. Bonus: magnitude pruning

Pruning removes redundant weights. Unstructured pruning (zeroing individual weights) mainly shrinks the model on disk after compression and rarely speeds up dense GPU matmuls by itself; *structured* pruning (removing whole channels/filters) is what actually reduces FLOPs and latency. We demo both so students see the difference.

```
1 import torch.nn.utils.prune as prune
2 import copy
3
4 pruned_model = copy.deepcopy(model).cpu().eval()
5
6 # Unstructured (per-weight) pruning -- shrinks storage, NOT latency on dense GPU kernels
```

```

7 for module in pruned_model.modules():
8     if isinstance(module, torch.nn.Conv2d):
9         prune.l1_unstructured(module, name="weight", amount=0.3)
10        prune.remove(module, "weight")
11
12 sparsity = sum((m.weight == 0).sum().item() for m in pruned_model.modules() if isinstance(m, torch.nn.Conv2d))
13 total = sum(m.weight.numel() for m in pruned_model.modules() if isinstance(m, torch.nn.Conv2d))
14 print(f"Unstructured sparsity in conv layers: {sparsity/total:.1%}")
15 print("Note: re-export this pruned model to ONNX + run it through Section 6/7 "
16       "to see it does NOT meaningfully speed up dense inference -- the lesson is "
17       "'sparsity != speed' unless using structured pruning or sparse-aware runtimes.")
18

```

✓ 9. Hardware-accelerated inference: TensorRT & OpenVINO

- **TensorRT** (NVIDIA): compiles the ONNX graph into a hardware-specific engine with kernel fusion, layer auto-tuning, and FP16/INT8 kernels tuned for the exact GPU. Biggest wins on NVIDIA GPUs/Jetson edge devices.
- **OpenVINO** (Intel): equivalent toolkit for Intel CPUs/iGPUs/VPUs — common in edge deployments on Intel-based hardware (retail kiosks, industrial cameras).

Colab's free GPU runtime doesn't ship TensorRT or OpenVINO by default, so below are runnable **install + compile stubs**. Uncomment to run; expect a few minutes for the TensorRT install/build.

```

1 # --- TensorRT via ONNX Runtime's TensorrtExecutionProvider ---
2 # Uncomment to actually install/run in this Colab session (adds ~3-5 min):
3 #
4 # !pip -q install tensorrt onnxruntime-gpu --extra-index-url https://pypi.nvidia.com
5 #
6 # trt_sess = ort.InferenceSession(
7 #     onnx_path,
8 #     providers=[
9 #         ("TensorrtExecutionProvider", {
10 #             "trt_fp16_enable": True,
11 #             "trt_engine_cache_enable": True,
12 #             "trt_engine_cache_path": "./trt_cache",
13 #         }),
14 #         "CUDAExecutionProvider",
15 #     ],
16 # )
17 # print(trt_sess.get_providers())

```

```

18 # # First call triggers engine build (slow); subsequent calls use the cached engine (fast).
19 # trt_sess.run(None, {"input": x_np})
20
21 print("TensorRT EP stub ready -- uncomment the block above on a CUDA-enabled runtime to build "
22       "and benchmark a real TensorRT engine, then add its result row to bench_df in Section 7.")
23

```

```

1 # --- OpenVINO (Intel CPU/iGPU) ---
2 # Uncomment on a CPU runtime (Runtime > Change runtime type > CPU) since OpenVINO targets Intel HW:
3 #
4 # !pip -q install onnxruntime-openvino openvino
5 #
6 # ov_sess = ort.InferenceSession(
7 #     onnx_path,
8 #     providers=[("OpenVINOExecutionProvider", {"device_type": "CPU_FP32"})],
9 # )
10 # print(ov_sess.get_providers())
11
12 print("OpenVINO EP stub ready -- best demoed on a CPU runtime (it targets Intel CPUs/VPUs, "
13       "the common case for edge cameras/kiosks rather than NVIDIA-GPU clouds).")
14

```

Classroom takeaway: TensorRT/OpenVINO sit *after* ONNX export and *alongside* quantisation — they're compilers that fuse ops and pick hardware-tuned kernels, often combined with FP16/INT8 for the largest latency wins. The benchmarking harness in Section 7 is exactly what you'd extend with a `trt_sess`/`ov_sess` row to make the comparison apples-to-apples.

✓ 10. Serving architecture patterns

A. ONNX Runtime REST API (lightweight microservice)

A minimal FastAPI wrapper around the `InferenceSession` from Section 5 — appropriate for a single-model service, low-to-moderate QPS, simple ops.

```

1 rest_api_code = '''
2 # save as app.py, run with: uvicorn app:app --host 0.0.0.0 --port 8000
3 from fastapi import FastAPI, UploadFile
4 import onnxruntime as ort
5 import numpy as np

```

```

6 from PIL import Image
7 import io
8
9 app = FastAPI()
10 sess = ort.InferenceSession("efficientnet_b0_int8_static.onnx",
11                             providers=["CUDAExecutionProvider", "CPUExecutionProvider"])
12 input_name = sess.get_inputs()[0].name
13
14 def preprocess(img: Image.Image) -> np.ndarray:
15     img = img.resize((224, 224)).convert("RGB")
16     x = (np.asarray(img).astype(np.float32) / 255.0 - 0.485) / 0.229
17     return x.transpose(2, 0, 1)[None, ...].astype(np.float32)
18
19 @app.post("/predict")
20 async def predict(file: UploadFile):
21     img = Image.open(io.BytesIO(await file.read()))
22     logits = sess.run(None, {input_name: preprocess(img)})[0]
23     return {"class_id": int(logits.argmax()), "confidence": float(logits.max())}
24 '''
25 print(rest_api_code)
26

```

✓ B. NVIDIA Triton Inference Server (production-grade multi-model serving)

Appropriate when you need: multiple models/versions, dynamic batching, concurrent model execution across GPUs, gRPC+HTTP, built-in metrics. Triton just needs a model repository folder with a `config.pbtxt` — no custom server code to write or maintain.

```

1 triton_config = '''
2 # model_repository/efficientnet_b0/config.pbtxt
3 name: "efficientnet_b0"
4 platform: "onnxruntime_onnx"
5 max_batch_size: 32
6 input [
7   { name: "input", data_type: TYPE_FP32, dims: [3, 224, 224] }
8 ]
9 output [
10  { name: "logits", data_type: TYPE_FP32, dims: [1000] }
11 ]
12 dynamic_batching {
13   preferred_batch_size: [8, 16, 32]
14   max_queue_delay_microseconds: 5000

```

```
15 }
16 instance_group [
17   { count: 2, kind: KIND_GPU }
18 ]
19 '''
20 print(triton_config)
21 print("\nDirectory layout expected by Triton:")
22 print("model_repository/efficientnet_b0/1/model.onnx")
23 print("model_repository/efficientnet_b0/config.pbtxt")
24 print("\nLaunch: docker run --gpus=all -p8000:8000 -p8001:8001 -p8002:8002 \\")
25 print("  -v$(pwd)/model_repository:/models nvcr.io/nvidia/tritonserver:24.05-py3 \\")
26 print("  tritonserver --model-repository=/models")
27 -
```

11. Edge vs Cloud deployment: design exercise

Dimension	Edge (e.g. Jetson, mobile, OpenVINO box)	Cloud API (e.g. Triton/ORT on GPU instance)
Latency	Lowest (no network round-trip); bounded by weak local compute	Higher network latency, but stronger compute per request
Cost model	Upfront hardware cost, no per-inference cloud bill	Pay-per-use/instance-hour; scales with traffic
Scalability	Fixed per device; scale = ship more devices	Elastic — autoscale GPU pool with demand
Model size budget	Tight (RAM/storage/power) → favours INT8/pruned small backbones	Looser — can afford larger backbones, batch heavily
Connectivity	Works offline	Requires network connectivity
Update/rollback	Harder (fleet OTA updates)	Easy (redploy a container)
Data privacy	Data never leaves device	Data transits network/cloud

Worked scenario for students to extend